E-Commerce Order and Inventory Database System

MySQL Database Project Documentation

Project Type	SQL / MySQL Database Project
Prepared By	Cherukuri Harshith Sai
Tools Used	MySQL, MySQL Workbench, EER Diagram Tool
Repository Files	schema.sql, data.sql, Reports.sql, EER_Diagram.png

Note: This is a database-only project. It focuses on relational database design, SQL querying, reporting, stored procedures, views, and triggers.

1. Introduction

This project is a MySQL-based relational database system designed for an e-commerce business. It manages customers, products, orders, order items, payments, and inventory updates. The project demonstrates database design, normalization, SQL reporting, stored procedures, views, and triggers using a realistic online retail workflow.

2. Objectives

- Design a normalized relational database for an online retail workflow
- Store and manage customer, product, order, order item, and payment data
- Use primary keys, foreign keys, constraints, and indexes to maintain data integrity
- Create business reports for revenue, customers, products, payments, and inventory
- Implement triggers to validate stock and automatically update product inventory
- Create stored procedures and views to simplify recurring database operations

3. Tools and Technologies Used

Tool / Technology	Purpose
MySQL	Database creation, querying, triggers, views, and stored procedures
MySQL Workbench	Running SQL scripts and generating the EER diagram
GitHub	Project hosting and version control
SQL	DDL, DML, joins, aggregations, constraints, and business reports

4. Database Design

The database follows a normalized relational design. The schema separates customer details, product inventory, order headers, order items, and payments into different tables. This reduces redundancy and makes the database easier to maintain.

4.1 Tables Used

Table	Purpose	Important Columns
Customers	Stores customer contact and location details	customer_id, first_name, last_name, email, phone, city, state

Products	Stores product details, price, category, and available stock	product_id, name, category, price, stock_quantity
Orders	Stores order-level details and links each order to a customer	order_id, customer_id, order_date, status
Order_Items	Stores products purchased in each order and resolves the many-to-many relationship between orders and products	order_item_id, order_id, product_id, quantity, price_per_item
Payments	Stores payment details for orders	payment_id, order_id, payment_date, amount, payment_method, status

4.2 Relationships

• One customer can place many orders (1:N relationship)

• One order can contain many order items (1:N relationship)

• One product can appear in many order items (1:N relationship)

• Each payment is linked to exactly one order (1:1 relationship)

• Order_Items acts as a junction table resolving the many-to-many relationship between Orders and Products

5. EER Diagram

The Enhanced Entity-Relationship (EER) diagram below visualizes the database structure, showing all tables, their attributes, primary keys, foreign keys, and relationships.

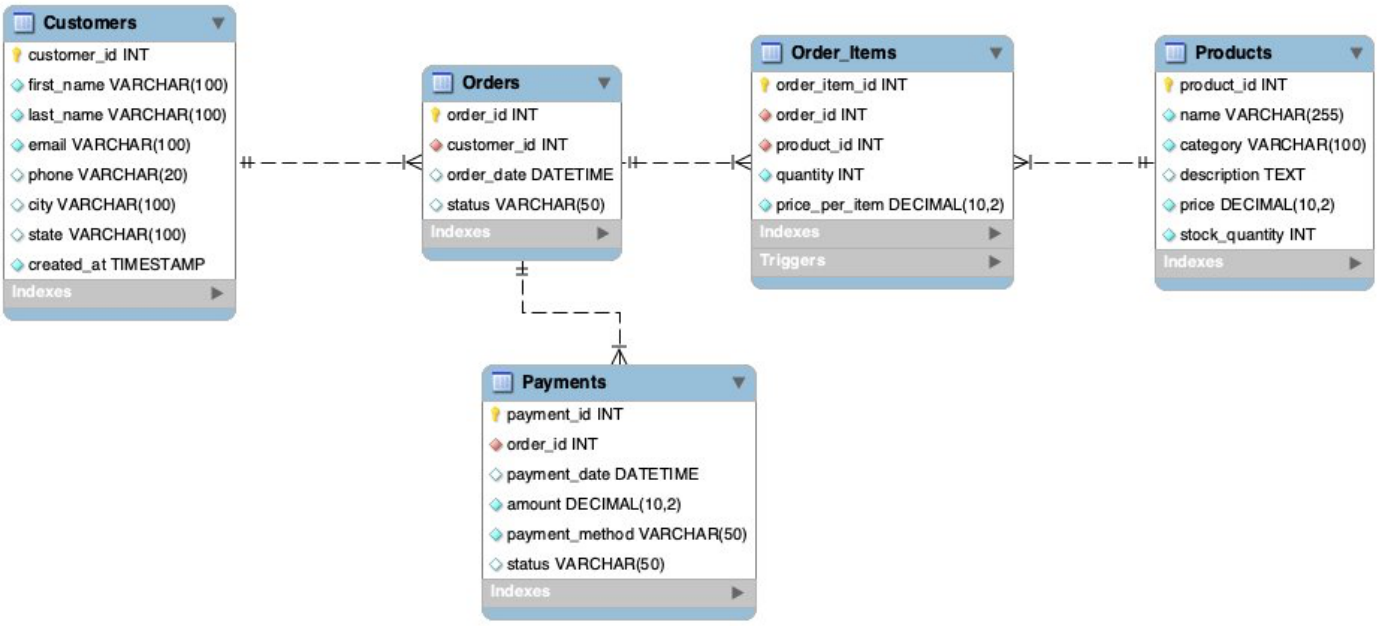

Figure 1: Database EER Diagram showing relationships between tables

6. Key Features and Implementation

Normalized Schema: The database uses separate tables for customers, products, orders, order items, and payments to reduce redundancy and improve data integrity.

Constraints and Keys: Primary keys, foreign keys, unique email constraint, CHECK constraints, and indexes are used to maintain valid data and improve query performance.

Expanded Sample Data: The project includes realistic sample data for customers, products, orders, order items, and payments so that business reports return meaningful results.

Views: Views such as v_SalesSummary and v_OrderTotals simplify complex joins and make reporting easier.

Stored Procedures: Stored procedures are used to retrieve customer order history and monthly sales information.

Triggers: Triggers validate stock before order items are inserted and automatically reduce product stock after a sale is recorded.

7. Business Reports Generated

The Reports.sql file includes queries that answer practical business questions, such as:

• Total revenue generated by completed payments

• Monthly sales trend based on order dates

- Top customers by total spending
- Best-selling products by quantity and revenue
- Category-wise revenue summary
- Low-stock products that may need restocking
- Payment method-wise revenue and count
- Average order value
- Repeat customers
- Order status summary

8. How to Run the Project

Run the files in the following order:

1. schema.sql - Creates the database, tables, constraints, indexes, and triggers

2. data.sql - Inserts sample data

3. Reports.sql - Creates views, stored procedures, and executes business reports

Recommended MySQL Workbench workflow:

1. Open MySQL Workbench and connect to your MySQL server

2. Open and execute schema.sql

3. Open and execute data.sql

4. Open and execute Reports.sql

9. Project Outcomes

The final database system successfully demonstrates relational database design and SQL implementation for a small e-commerce workflow. The project supports data storage, order tracking, payment records, sales analysis, and automated inventory updates.

- Database tables are connected using proper foreign key relationships
- Sample data supports meaningful sales and inventory analysis
- Reports help analyze monthly revenue, top customers, best-selling products, and low-stock items
- Triggers enforce inventory rules and reduce manual stock updates
- Stored procedures and views make repeated reporting tasks easier

10. Conclusion

This project demonstrates database design and SQL development skills using MySQL. It shows how a relational database can be used to manage e-commerce data, generate useful business reports, and automate inventory updates through triggers. The project is suitable as a database-focused portfolio project and can be extended in the future with a backend API or dashboard after learning backend development.

Appendix: Repository Files

- schema.sql - Database schema, constraints, indexes, and triggers
- data.sql - Sample data for all tables
- Reports.sql - Business reports, views, and stored procedures
- EER_Diagram.png - Visual database schema diagram
- README.md - GitHub project documentation
- E-Commerce_Order_and_Inventory_Database_System.pdf - Complete project documentation